



Construir software en la nube mientras duermes

Evaluación con la suite de Claude

8 de octubre de 2026

ÍNDICE

Contenido

- 01** PC encendido vs nube
 - 02** Sesión cloud vs routine
 - 03** Agentes: qué son, para qué sirven y cómo se construyen
 - 04** PR o pull request
 - 05** Harness: qué es, para qué sirve y cómo se implementa
 - 06** Escalera de progreso: los 5 niveles
-

PARTE 1

PC encendido vs nube

PREGUNTA

Necesito que mientras duerma, se pueda estar construyendo un software, necesito evaluar si es posible con la suite de Claude, y tener un cuadro comparativo entre usar un PC encendido vs en la nube.

Sí, es posible, y hay dos formas reales de hacerlo con Claude Code: dejar una sesión corriendo en tu PC o mandarla a una sesión en la nube de Anthropic. Para trabajar mientras duermes, la nube es la opción más confiable, siempre que tu proyecto esté en GitHub.

Cuadro comparativo

Criterio	PC encendido (sesión local)	Nube (sesión cloud / routine)
Dónde corre	En tu máquina	En una VM aislada de Anthropic (4 vCPU, 16 GB RAM, 30 GB de disco)
¿Necesita tu PC?	Sí: encendido, despierto y con la app o terminal abierta	No, sigue aunque apagues el PC
Qué lo puede cortar de noche	Suspensión, reinicio por Windows Update, corte de luz o internet, cerrar la app	Que Claude quede esperando una respuesta tuya: la VM se pausa y el trabajo en curso puede perderse
Permisos	Si falta una aprobación, se queda detenido hasta que respondas	Las routines corren sin pedir aprobación; en sesiones cloud eliges el modo de permisos
Acceso a tus archivos	Todo lo local: archivos fuera del repo, <code>.env</code> , bases de datos y herramientas instaladas	Solo un clon nuevo del repo; lo demás se configura (variables, script de instalación)
Entorno	El que ya tienes	Trae Docker, PostgreSQL y Redis; red limitada por defecto a una lista de dominios confiables
Requisitos	Plan de pago y la app o CLI	Plan Pro, Max, Team o Enterprise, y GitHub conectado
Programar horario	Tareas programadas locales (mínimo cada 1 minuto); si el PC duerme, esa ejecución se salta	Routines (mínimo cada 1 hora), también por API o eventos de GitHub; están en vista previa
Supervisar desde el celular	Sí, con Remote Control, mientras el PC siga vivo	Sí, desde claude.ai/code o la app móvil
Resultado al despertar	Cambios en tu carpeta local	Una rama <code>claude/...</code> lista para revisar y abrir un pull request
Costo	Tu plan, más electricidad y desgaste del PC	Tu plan; la VM no se cobra aparte
Ideal para	Proyectos que dependen de cosas que solo existen en tu PC	Trabajo nocturno desatendido sobre un repo en GitHub

Lo que debes saber antes de decidir

- **El límite real es el uso de tu plan.** En ambos casos se consume la misma cuota; si se agota a mitad de la noche, el trabajo se detiene hasta que se renueve.
- **Despiertas con trabajo para revisar, no con un producto terminado.** Funciona bien con tareas acotadas, una especificación clara y pruebas automáticas que Claude pueda correr para verificarse solo.
- **Un estado “verde” en una routine no garantiza éxito.** Solo indica que la sesión terminó sin error de infraestructura; hay que abrirla y revisar qué hizo.
- **En tu PC con Windows 10 Home,** activa “Mantener el equipo despierto” en la configuración de la app y pausa las actualizaciones automáticas esa noche.

Recomendación

Usa la nube para el trabajo nocturno: planifica conmigo de día, guarda el plan en el repo y lanza la ejecución en una sesión cloud antes de dormir. Deja el PC encendido solo cuando la tarea necesite algo que no puedas subir al repo.

Si me dices qué proyecto quieres construir, te dejo configurada la primera ejecución nocturna.

FUENTES

- [Claude Code en la nube](#)
- [Entornos cloud](#)
- [Routines](#)
- [Tareas programadas en Desktop](#)
- [Remote Control](#)

PARTE 2

Sesión cloud vs routine

PREGUNTA

En el contexto de trabajar en la nube, dame un cuadro comparativo entre: sesión cloud vs routine.

Una routine no es algo distinto de una sesión cloud: es una configuración guardada (instrucciones, repositorios y conectores) que **lanza sesiones cloud automáticamente**. Ambas usan la misma VM, el mismo entorno y la misma cuota de tu plan; cambia quién la inicia y cuánta supervisión admite.

Criterio	Sesión cloud	Routine
Cómo arranca	La lanzas tú, a mano, cuando quieres	Sola: por horario, por llamada API o por un evento de GitHub (PR o release)
Frecuencia	Una vez, la que iniciaste	Recurrente (cada hora, diaria, días hábiles, semanal) o una sola vez en una fecha futura; intervalo mínimo de 1 hora
Instrucciones	Las escribes en el momento y puedes corregir sobre la marcha	Fijas y guardadas; deben ser autosuficientes porque nadie las aclara durante la ejecución
Permisos	Eliges el modo de permisos al crearla y puedes cambiarlo	Sin selector: corre de forma autónoma, sin pedir aprobación
Interacción	Conversación viva: puedes guiarla desde el celular o el navegador	Pensada para correr sin ti; después puedes abrir la ejecución y continuar la conversación
Memoria entre ejecuciones	Una sola conversación con todo su contexto	Cada ejecución es una sesión nueva; no recuerda la anterior
Punto de partida del código	La rama que elijas	Un clon nuevo desde la rama por defecto en cada ejecución
Conectores (Slack, etc.)	Los de tu cuenta	Todos incluidos por defecto y usables sin preguntar; conviene quitar los que no necesite
Resultado	Rama <code>claude/...</code> y pull request	Lo mismo, una sesión por cada ejecución
Señal de éxito	Lo ves en la conversación	El estado “verde” solo indica que terminó sin error de infraestructura; hay que abrir la ejecución
Madurez	Función estable	Vista previa de investigación; límites y comportamiento pueden cambiar
Dónde se crea	<code>claude.ai/code</code> , app móvil, app de escritorio (opción Cloud) o <code>claude --cloud</code>	<code>claude.ai/code/routines</code> , app de escritorio (Routines) o <code>/schedule</code>
Ideal para	Una tarea grande y puntual que quieres dejar corriendo esta noche	Trabajo repetible que debe ocurrir todas las noches sin que hagas nada

Cuál usar para construir mientras duermes

Empieza con una **sesión cloud**: defines el plan de día, la lanzas antes de dormir y revisas la rama al despertar. Es más simple y puedes corregirla si se desvía.

Pasa a una **routine** cuando el trabajo se vuelva repetitivo, por ejemplo una nocturna que tome la siguiente tarea pendiente, la implemente y abra un pull request. Como cada ejecución empieza sin memoria, la lista de tareas y el avance deben estar guardados en el propio repositorio.

FUENTES

- [Claude Code en la nube](#)
- [Routines](#)

PARTE 3

Agentes: qué son, para qué sirven y cómo se construyen

PREGUNTA

En el contexto del trabajo en la nube, enséñame qué es un agente, para qué sirve, cómo se construyen. Y dame 2 cuadros comparativos: uso de agentes vs sesión cloud y uso de agentes vs routine. Después de estos cuadros comparativos dame tu recomendación y un ejemplo práctico y fácil de entender.

Tomo “agente” en el sentido que aplica a Claude Code en la nube: los **subagentes**, especialistas que tú defines y que Claude usa dentro de una sesión. (Existe otro camino, el Agent SDK, para programar tus propios agentes como producto; requiere desarrollo y no es lo que necesitas aquí.)

Qué es un agente

Un agente es Claude con tres cosas: un rol (instrucciones propias), un conjunto de herramientas permitidas y la capacidad de trabajar solo hasta terminar un encargo. El Claude que conduce una sesión cloud ya es un agente; los que tú construyes son ayudantes especializados a los que él les delega partes del trabajo.

Las tres piezas no compiten entre sí, son capas:

- **Routine:** *cuándo* arranca el trabajo.
- **Sesión cloud:** *dónde* ocurre.
- **Agente:** *quién* hace cada parte.

Para qué sirve

- **Especializar:** un revisor que solo revisa, un probador que solo prueba.
- **Limitar riesgos:** puedes darle a un agente solo permisos de lectura.
- **Mantener el foco:** cada agente trabaja en su propio contexto y devuelve solo un resumen, así la conversación principal no se llena de ruido.
- **Trabajar en paralelo:** varios agentes pueden correr a la vez dentro de una sesión.
- **Ahorrar cuota:** las tareas simples pueden ir a un modelo más económico.

Cómo se construye

Un agente es un archivo de texto (Markdown) dentro de la carpeta `.claude/agents/` de tu repositorio. Lo más fácil es pedirle a Claude que lo cree; también puedes escribirlo a mano:

```
---
name: revisor
description: Revisa los cambios antes de abrir un pull request. Úsalo siempre al terminar una tarea.
tools: Read, Grep, Glob
model: sonnet
---
```

Eres un revisor de código. Revisa los archivos que te indiquen y entrega una lista de errores, riesgos de seguridad y casos sin probar. No modifiques archivos.

Solo `name` y `description` son obligatorios. La descripción es clave: con ella Claude decide cuándo delegarle.

Para la nube, el archivo debe estar **guardado en el repo** (commit y push). Los agentes que solo existen en tu PC no llegan a la sesión cloud.

Cuadro 1: agente vs sesión cloud

Criterio	Agente	Sesión cloud
Qué es	Un especialista con rol y herramientas propias	El lugar de trabajo: una VM en la nube con tu repo
¿Funciona solo?	No, necesita una sesión que lo invoque	Sí
Cómo se crea	Archivo en <code>.claude/agents/</code> del repo	Se lanza desde <code>claude.ai/code</code> , la app o <code>claude --cloud</code>
Contexto	Propio y limpio: recibe el encargo y devuelve un resumen	Una conversación principal que acumula todo
Permisos	Restringibles por agente (por ejemplo, solo lectura)	Los de la sesión completa
Modelo	Puedes elegir uno distinto por agente	El de la sesión
Paralelismo	Varios a la vez dentro de una sesión	Una por tarea; puedes lanzar varias sesiones
Duración	Permanente: vive en el repo y se reutiliza	Termina con la tarea; queda el historial
Riesgo nocturno	Si la VM se recicla, el trabajo de agentes en curso se pierde	Se pausa si queda esperando tu respuesta

Cuadro 2: agente vs routine

Criterio	Agente	Routine
Qué es	Un especialista	Un disparador guardado que lanza sesiones cloud solas
Qué define	Rol, reglas, herramientas y modelo	La tarea, los repos, el entorno, los conectores y el disparador
Cómo arranca	Lo invoca Claude dentro de una sesión, por su descripción o porque se lo pides	Por horario, llamada API o evento de GitHub
¿Funciona solo?	No	Sí
Dónde vive	En el repositorio	En tu cuenta de claude.ai
Memoria	Empieza limpio en cada encargo	Cada ejecución es una sesión nueva
Permisos	Restringibles por agente	Corre sin pedir aprobación
Madurez	Función estable	Vista previa de investigación
Relación	Una routine puede usar los agentes del repo	Es quien pone a trabajar a los agentes cada noche

Recomendación

No elijas entre ellos: combínalos, pero en este orden.

- 1. Empieza solo con una sesión cloud.** No necesitas construir ningún agente para arrancar.
- 2. Agrega uno o dos agentes** cuando veas un problema repetido de calidad. Los más útiles son un revisor de solo lectura y un probador.
- 3. Crea la routine al final**, cuando el flujo ya funcione bien a mano y quieras que se repita cada noche.

Evita crear muchos agentes desde el inicio: cada uno consume cuota de tu plan y complica el diagnóstico cuando algo falla.

Ejemplo práctico

Piensa en una obra. La routine es la alarma que abre la obra a las 11 pm, la sesión cloud es el taller con las herramientas, Claude es el maestro de obra, y los agentes son el inspector y el probador que él llama cuando los necesita.

Aplicado a un sistema de cotizaciones:

1. En el repo hay un archivo `TAREAS.md` con la lista de pendientes, por ejemplo “agregar descuento por volumen”.
2. En `.claude/agents/` hay dos agentes: `probador` (corre las pruebas y reporta fallas) y `revisor` (el de arriba).
3. Una routine diaria a las 11 pm tiene estas instrucciones: “Toma la primera tarea pendiente de `TAREAS.md` e impleméntala. Usa el agente probador para correr las pruebas y el agente revisor para revisar los cambios. Corrige lo que encuentren, marca la tarea como hecha y abre un pull request.”
4. Al despertar tienes un pull request con el descuento por volumen implementado, probado y revisado, listo para que lo apruebes.

FUENTES

- [Subagentes](#)
- [Entornos cloud](#)
- [Routines](#)

PARTE 4

PR o pull request

PREGUNTA

¿Qué es un PR, qué es un pull request?

PR es la abreviatura de **pull request**: son lo mismo. Es una solicitud para incorporar cambios al código principal de un proyecto, que queda en espera hasta que alguien la revise y la apruebe.

Cómo funciona

1. El código oficial del proyecto vive en la rama principal (normalmente llamada `main`).
2. Quien va a hacer cambios trabaja en una **rama**, que es una copia separada donde nada de lo que haga afecta al código oficial.
3. Al terminar, abre un pull request en GitHub: “estos son mis cambios, pido que se incorporen”.
4. Tú revisas: GitHub muestra línea por línea qué se agregó y qué se quitó.
5. Si te convence, lo apruebas y se **fusiona** (merge) con el código principal. Si no, pides correcciones o lo cierras, y el código oficial queda intacto.

Una comparación sencilla

Es como cuando un empleado te entrega el borrador de un contrato con los cambios marcados. El contrato vigente no cambia hasta que tú lo lees y lo firmas.

Por qué importa para trabajar mientras duermes

Es tu punto de control. Claude trabaja de noche en su propia rama (`claude/...`) y te deja un pull request; nada entra a tu software real hasta que tú lo apruebas en la mañana. Si lo que hizo está mal, lo descartas con un clic y no pasó nada.

PARTE 5

Harness: qué es, para qué sirve y cómo se implementa

PREGUNTA

¿Y qué es un harness, para qué sirve y cómo se implementa?

Un **harness** (en español, “arnés”) es el programa que rodea al modelo de IA y lo convierte en un agente capaz de trabajar. El modelo por sí solo únicamente lee texto y responde texto; el harness es lo que le da manos, reglas y memoria de trabajo.

Claude Code es un harness. El modelo es el motor; el harness es el resto del carro: volante, frenos, tablero y carrocería.

Para qué sirve

- **Ejecuta el ciclo de trabajo:** el modelo pide una acción, el harness la ejecuta, le devuelve el resultado y repite hasta terminar la tarea.
- **Le da herramientas:** leer y editar archivos, correr comandos, buscar en la web, conectarse a servicios como GitHub o Slack.
- **Aplica los permisos:** decide qué puede hacer solo, qué requiere tu aprobación y qué está prohibido.
- **Administra el contexto:** carga las instrucciones del proyecto y resume la conversación cuando se alarga.
- **Coordina:** lanza subagentes, sesiones cloud y tareas programadas.

Por eso el mismo modelo rinde distinto según el harness: la calidad del trabajo nocturno depende tanto de cómo está configurado como del modelo.

Cómo se implementa

Hay tres niveles, de menor a mayor esfuerzo:

1. **Usar uno existente y configurarlo.** Es tu caso: Claude Code ya está construido, solo lo ajustas a tu proyecto.
2. **Armar uno propio con el Agent SDK.** Anthropic te da las piezas de Claude Code para construir tu propio agente con código. Sirve para crear un producto, por ejemplo un asistente dentro de tu software.
3. **Construirlo desde cero** contra la API de Claude. Programas tú el ciclo, las herramientas y los permisos. Solo se justifica con necesidades muy particulares.

Qué significa “configurarlo” en tu caso

Todo esto son archivos en tu repositorio, así que viajan a la sesión cloud:

Pieza	Archivo	Para qué
Instrucciones del proyecto	CLAUDE.md	Explica qué es el software, cómo se corre, cómo se prueba y qué reglas seguir
Permisos	.claude/settings.json	Define qué puede hacer sin preguntar y qué tiene prohibido
Hooks	.claude/settings.json	Acciones automáticas, por ejemplo correr las pruebas después de cada cambio
Agentes	.claude/agents/	Los especialistas que vimos antes
Skills	.claude/skills/	Procedimientos reutilizables, paso a paso
Conexiones externas	.mcp.json	Acceso a servicios y datos fuera del repo

Para empezar basta con un buen CLAUDE.md y pruebas automáticas. Con esas dos piezas Claude sabe qué hacer y puede comprobar solo si lo hizo bien, que es lo que más pesa cuando nadie lo está supervisando.

Una aclaración: fuera del mundo de agentes, “test harness” también se usa para el programa que corre las pruebas de un software. Es la misma idea aplicada a otra cosa: la estructura que rodea y controla algo.

PARTE 6

Escalera de progreso: los 5 niveles

PREGUNTA

Si quisiera trabajar en la nube, mientras duermo, como escalera de valor o progreso, ¿cuáles serían los 5 niveles, para lograr todo lo conversado antes?

Cada nivel entrega valor por sí mismo y prepara el siguiente. La regla es no subir hasta cumplir la señal del nivel actual.

Nivel	Nombre	Qué haces	Qué obtienes	Señal para subir
1	Cimientos	Subes el proyecto a GitHub, conectas GitHub con Claude y lanzas de día una sesión cloud con una tarea pequeña, mirándola trabajar	Tu primer pull request, y entiendes el ciclo rama → revisión → aprobación	Aprobaste un PR hecho por Claude en la nube
2	Harness mínimo	Agregas al repo un <code>CLAUDE.md</code> (qué es el software, cómo se corre, cómo se prueba) y pruebas automáticas	Claude deja de hacerte preguntas y puede comprobar solo si lo hizo bien	Una sesión termina una tarea de principio a fin sin que intervengas
3	Primera noche	De día escribes el plan y la lista <code>TAREAS.md</code> ; antes de dormir lanzas una sesión cloud; en la mañana revisas	Trabajo real hecho mientras duermes	Tres noches seguidas despiertas con un PR útil
4	Especialistas	Creas dos agentes en <code>.claude/agents/</code> : un probador y un revisor de solo lectura	Menos errores y menos correcciones tuyas en la mañana	Apruebas la mayoría de los PR casi sin cambios
5	Piloto automático	Creas una routine nocturna que toma la siguiente tarea de <code>TAREAS.md</code> , usa los agentes y abre el PR	Ya no lanzas nada: solo mantienes la lista y apruebas	El sistema corre una semana sin que lo toques

Qué cambia en tu rol

- **Niveles 1 y 2:** aprendes y preparas. Aquí está la mayor parte del esfuerzo.
- **Nivel 3:** pasas de programar a encargar.
- **Niveles 4 y 5:** pasas de encargar a dirigir: defines qué construir y apruebas resultados.

Tres advertencias

- **El nivel 2 es el que más se salta y el que más pesa.** Sin pruebas automáticas, Claude no tiene cómo saber si rompió algo, y los niveles siguientes producen PR que no puedes aprobar con confianza.
- **El techo es la cuota de tu plan.** Agentes y routines consumen más; si se agota a mitad de la noche, el trabajo se detiene.
- **Tu aprobación no se automatiza.** En todos los niveles, nada entra al software real sin que tú apruebes el PR.

Dime con qué proyecto quieres empezar y arrancamos el nivel 1.